Workflow UI Redesign - Modern Flowchart Interface

Overview

The workflow pages have been completely redesigned with a sleek, modern, animated flowchart interface that aligns with ReelForge's brand identity. This redesign introduces:

1. **Interactive Flowchart Builder** - Drag-and-drop nodes with real-time visualization
2. **Real-Time Execution Viewer** - Live SSE-powered execution monitoring with animated flow
3. **Custom Node Types** - Beautiful, gradient-styled nodes for Agent, Conditional, ForEach, and ReviewLoop steps
4. **Enhanced Security** - Execution-scoped SSE endpoints with proper authorization

---

What's New

1. Flowchart-Based Workflow Builder

Components:

- `FlowchartBuilder.tsx` - Main flowchart component using @xyflow/react
- `nodes/AgentNode.tsx` - Violet gradient, AI agent step node
- `nodes/ConditionalNode.tsx` - Orange gradient, branching logic node
- `nodes/ForEachNode.tsx` - Cyan gradient, loop iteration node
- `nodes/ReviewLoopNode.tsx` - Pink gradient, quality gate node (special emphasis)

Features:

- ✨ Fully draggable nodes with smooth animations
- 🎨 Color-coded nodes by type with gradient backgrounds
- 🔄 Auto-layout button for vertical arrangement
- ⚙️ Expandable configuration panels on hover
- 🔗 Animated edges showing workflow flow
- ➕ Modal-based step type selection

Usage:

```
import { FlowchartBuilderWrapper } from
'@/components/workflows/FlowchartBuilder';

<FlowchartBuilderWrapper steps={steps} onChange={setSteps} />
```

2. Real-Time Execution Viewer

Location: `/projects/[id]/workflows/[workflowId]/executions/[executionId]`

Features:

- 🔴 **Live SSE Updates** - Real-time event streaming for single execution
- 📊 **Progress Tracking** - Visual progress bar and completion stats
- 🎯 **Animated Flow Visualization** - Nodes change color based on status:
  - 🟢 **Green** - Completed
  - 🔵 **Blue** - Running (with pulse animation)
  - 🔴 **Red** - Failed
  - ⚪ **Gray** - Pending
- ⏱️ **Event Timeline** - Chronological list of workflow events with animations
- 📈 **Execution Metrics** - Duration, iterations, step counts

SSE Events Supported:

- `connected` - Initial connection confirmation
- `execution.completed` - Workflow finished successfully
- `execution.failed` - Workflow failed with error
- `step.completed` - Individual step completed

3. Backend Enhancements

New Endpoint:

```
GET
/api/v1/projects/:projectId/workflows/:workflowId/executions/:executionId/event
s
```

Security:

- ☑ Requires authentication
- ☑ Verifies project ownership OR admin status
- ☑ Filters events to ONLY the requested execution ID
- ☑ Prevents cross-execution data leaks

Implementation Location: `api/handlers/workflows.go`

Technical Stack

Frontend Dependencies

- **@xyflow/react** - Modern flowchart/diagram library (formerly ReactFlow)
- **framer-motion** - Smooth animations and transitions
- **@mantine/core v8** - UI component library

Key Technologies

- **Server-Sent Events (SSE)** - Real-time unidirectional streaming

- **React Hooks** - useState, useEffect, useCallback for state management
- **TypeScript** - Full type safety across components

Component Architecture

FlowchartBuilder

```
FlowchartBuilder
├── ReactFlow (canvas)
│   ├── Background (dots pattern)
│   ├── Controls (zoom, fit view)
│   └── Panel (add step, auto-layout buttons)
├── Custom Nodes
│   ├── AgentNode (violet)
│   ├── ConditionalNode (orange)
│   ├── ForEachNode (cyan)
│   └── ReviewLoopNode (pink)
└── AddStepModal (step type selector)
```

Execution Viewer

```
ExecutionDetailPage
├── Status Card (progress, metrics)
├── Flow Visualization (ReactFlow)
│   ├── Animated nodes (status-based colors)
│   └── Animated edges (pulse on running)
└── Event Timeline (real-time updates)
```

Node Types & Visual Design

1. Agent Node (Violet)

- **Purpose:** Execute AI agent with specific prompt
- **Color:** `linear-gradient(135deg, #8b5cf6 0%, #6d28d9 100%)`
- **Icon:** Robot
- **Config:** Agent picker, input mapping

2. Conditional Node (Orange)

- **Purpose:** Branch workflow based on condition
- **Color:** `linear-gradient(135deg, #f59e0b 0%, #d97706 100%)`
- **Icon:** Git Branch
- **Config:** Condition expression, true/false branches
- **Handles:** Two source handles (true/false)

3. ForEach Node (Cyan)

- **Purpose:** Iterate over collection
- **Color:** `linear-gradient(135deg, #06b6d4 0%, #0891b2 100%)`
- **Icon:** Repeat
- **Config:** Loop source, max iterations, target step

4. Review Loop Node (Pink) ★ SPECIAL

- **Purpose:** Quality gate with score-based looping
- **Color:** `linear-gradient(135deg, #ec4899 0%, #db2777 100%)`
- **Icon:** Star
- **Config:** Min score (1-10), max iterations, loop target
- **Why Special:** Implements the ReviewAgent's quality assurance pattern with dedicated step type
- **Handles:** Two source handles (continue/loop)

Animation & Interaction Patterns

Hover Effects

- Nodes expand to show configuration on hover
- Smooth height transitions (0.2s ease)
- Card lift effect with translateY(-2px)

Pulse Animation

```
@keyframes pulse {
  0%, 100% {
    box-shadow: 0 0 0 0 rgba(59, 130, 246, 0.7);
  }
  50% {
    box-shadow: 0 0 0 10px rgba(59, 130, 246, 0);
  }
}
```

Applied to running nodes for visual feedback.

Edge Animation

- Animated stroke-dasharray on running steps
- Color-coded based on source/target status
- Smooth transitions on state changes

Framer Motion

```
<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.4 }}
>
```

Used for staggered card reveals and timeline events.

API Integration

Workflow API (`lib/api/workflows.ts`)

```
// New function added
export function getWorkflowExecution(
  projectId: string,
  workflowId: string,
  executionId: string
): Promise<WorkflowExecution>
```

SSE Connection Pattern

```
const eventSource = new EventSource(

`/api/v1/projects/${projectId}/workflows/${workflowId}/executions/${executionId
}/events`
);

eventSource.addEventListener('step.completed', (e) => {
  const event = JSON.parse(e.data);
  // Update UI
});
```

Usage Examples

Creating a New Workflow

1. Navigate to `/projects/{id}/workflows/new`
2. Enter workflow name
3. Click "Add Step" button in the flowchart
4. Select step type from modal
5. Configure each node by hovering and editing
6. Click "Create Workflow"

Editing a Workflow

1. Navigate to `/projects/{id}/workflows/{workflowId}`
2. Drag nodes to rearrange
3. Click nodes to edit configuration
4. Use "Auto Layout" to reset positions
5. Click "Save Changes"

Executing & Monitoring

1. Click "Execute" button on workflow page
2. Automatically redirected to execution viewer
3. Watch real-time progress with animated flow
4. See events appear in timeline as they occur
5. View final status and metrics when complete

Alignment with Backend Concepts

Agent Output Schemas

- Each agent node can reference an agent with a fixed output schema (via `agentDefinitionId`)
- Input mappings (`inputMappingJson`) use template syntax: `"{{steps.1.output}}"`
- Schema validation happens server-side in the WorkflowEngine

Review Agent Special Handling

- **ReviewLoop step type** maps directly to `ReviewLoopStepExecutor` in WorkflowEngine
- Enforces min score threshold (default: 9/10)
- Automatically loops back to specified step if score too low
- Max iterations prevents infinite loops
- Visual distinction (pink gradient, star icon, enhanced styling)

Step Type Mapping

Frontend StepType	Backend StepExecutor
`Agent`	`AgentStepExecutor`
`Conditional`	`ConditionalStepExecutor`
`ForEach`	`ForEachStepExecutor`
`ReviewLoop`	`ReviewLoopStepExecutor`

Performance Considerations

ReactFlow Optimization

- Memoized node components prevent unnecessary re-renders

- Controlled state updates via `useNodesState` / `useEdgesState`
- Lazy rendering for large workflows

SSE Efficiency

- Server-side event filtering (only sends relevant events)
- Client-side buffering with 32-event channel capacity
- 25-second keep-alive pings prevent proxy timeouts
- Automatic reconnection on connection loss

Animation Performance

- CSS transforms (translateY) over position changes
- GPU-accelerated animations via `will-change` hints
- Framer Motion's optimized motion engine

File Structure

```
web/
├── app/
│   └── (app)/projects/[id]/workflows/
│       ├── new/page.tsx                    (NEW: Flowchart builder)
│       └── [workflowId]/
│           ├── page.tsx                     (NEW: Flowchart editor)
│           └── executions/[executionId]/
│               └── page.tsx                 (NEW: Real-time viewer)
├── components/workflows/
│   ├── FlowchartBuilder.tsx                (NEW)
│   ├── AddStepModal.tsx                    (NEW)
│   ├── nodes/
│   │   ├── AgentNode.tsx                   (NEW)
│   │   ├── ConditionalNode.tsx             (NEW)
│   │   ├── ForEachNode.tsx                 (NEW)
│   │   └── ReviewLoopNode.tsx              (NEW)
│   ├── WorkflowStepList.tsx                (LEGACY - still used for types)
│   └── ExecutionHistory.tsx                (RETAINED)
└── lib/api/workflows.ts                    (UPDATED: added
getWorkflowExecution)

api/
├── handlers/
│   └── workflows.go                        (UPDATED: new SSE endpoint)
└── models/
    └── project.go                          (NEW: for ownership checks)
```

Future Enhancements

Potential Improvements

1. **Minimap** - Add ReactFlow minimap for large workflows
2. **Zoom Controls** - Enhanced zoom with wheel events
3. **Node Grouping** - Visual grouping of related steps
4. **Custom Edges** - Curved edges for complex branches
5. **Undo/Redo** - History stack for workflow edits
6. **Templates** - Pre-built workflow templates
7. **Execution Replay** - Step-by-step playback of past executions
8. **Performance Metrics** - Token usage, duration graphs
9. **Collaborative Editing** - Real-time multi-user editing
10. **Export/Import** - JSON export for version control

Known Limitations

- Maximum ~100 steps per workflow (ReactFlow performance)
- SSE requires modern browser (no IE11)
- Mobile experience not yet optimized
- No offline editing capability

Troubleshooting

Common Issues

Q: Flowchart not rendering
A: Ensure `@xyflow/react` styles are imported: `import '@xyflow/react/dist/style.css';`

Q: SSE connection failing
A: Check authentication cookie and project ownership. Ensure nginx is not buffering SSE responses.

Q: Nodes not draggable
A: Verify `nodesDraggable={true}` (default) on ReactFlow component.

Q: TypeScript errors on node types
A: Node components use `as any` cast for nodeTypes - this is intentional due to @xyflow/react type complexity.

Credits

Design: Modern, sleek, gradient-based aesthetic matching ReelForge brand
Icons: Tabler Icons (@tabler/icons-react)
Flow Library: @xyflow/react (Svelte UG)
Animations: Framer Motion
UI Framework: Mantine v8

Conclusion

This redesign transforms the workflow experience from a basic form-based interface into a **professional, animated, visually stunning flowchart builder** that:

- ☑ Matches ReelForge's sleek brand identity
- ☑ Makes complex workflows intuitive and visual
- ☑ Provides real-time execution monitoring
- ☑ Aligns perfectly with backend concepts
- ☑ Emphasizes the ReviewAgent's special role
- ☑ Maintains security and performance

The workflow pages are now production-ready and deliver a world-class user experience. 🚀